

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of THURSDAY 4 JULY 2024 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

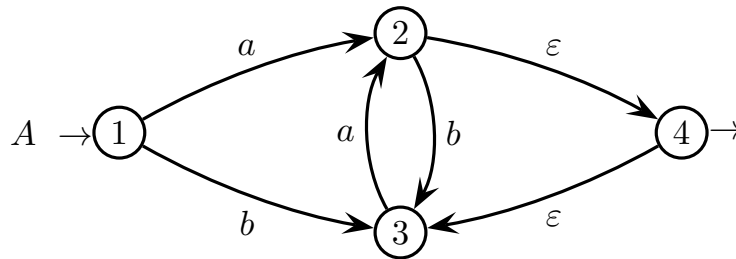
TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A below, over the two-letter alphabet $\Sigma = \{a, b\}$, with four states (initial state 1 and final state 4), which has spontaneous transitions (ε -transitions):



Answer the following questions:

- (a) Write all the valid strings of length ≤ 3 (less than or equal to three), and for those of length exactly three, list their computations.
 - (b) By applying the Berry-Sethi (BS) method to automaton A , find an equivalent deterministic automaton A_{det} . Examine if the deterministic automaton A_{det} is minimal or not, and if necessary minimize it.
 - (c) Cut all the spontaneous transitions from automaton A . Is the resulting automaton deterministic or not?
 - (d) By applying the Brzozowski-McCluskey (BMC) method to automaton A , find an equivalent regular expression R .
 - (e) (optional) Find a deterministic automaton $\overline{A}_{det}$ that recognizes the complement of language $L(A)$, and explain if such an automaton $\overline{A}_{det}$ is minimal or not.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the two languages L_1 and L_2 defined below, over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L_1 = \{ a^{2n} b^{n+k} \mid n > 0 \wedge k \geq 0 \}$$

$$L_2 = \{ a^{2n} b^k \mid n \geq 0 \wedge 0 \leq k \leq 2n \}$$

Answer the following questions:

- (a) With reference to the language L_1 , define a non-ambiguous BNF grammar G_1 that generates it and draw the syntax tree of the sample string:

$$a^2 b^3$$

- (b) With reference to the language L_2 , define a non-ambiguous BNF grammar G_2 that generates it and draw the syntax tree of the sample string:

$$a^6 b^3$$

- (c) (optional) Explain why grammar G_2 is not ambiguous (avoid generic arguments, and refer explicitly to the features of G_2).
-

2. A programming language, which can be seen as a simplified version of TypeScript, allows the definition of inductive custom data types such as:

```
type Tree = number | string | Tree [ ]
```

where symbol **Tree** is the identifier of the type to be declared, and symbols **number** and **string** are terminals that denote primitive types. Apart from **number** and **string**, one more primitive type is available: **boolean**. The notation **Tree []** denotes an *array* of elements of type **Tree**. The expression **number | string | Tree []** denotes a *union* non-primitive type where, in this case, the objects of that type can be numbers, strings or arrays of **Tree**. In addition to union and array, the language also allows a *tuple* type, which is constructed as in the following example:

```
type Student = [ string, string, number, number, boolean ]
```

where, e.g., the type **Student** is defined by their surname and name, two strings denoted by the primitive type **string**, their person code and enrollment code, both denoted by the primitive type **number**, and whether they have a currently active career, denoted by the primitive type **boolean**.

Answer the following questions:

- (a) Define a non-ambiguous EBNF grammar for the type definition statement. In the grammar, identifiers shall be denoted by the nonterminal $\langle \text{ID} \rangle$, which is assumed to be already defined elsewhere and does not need to be expanded.
- (b) Draw the parse tree of the sentence below:

```
type Tree = number | Tree [ ]
```

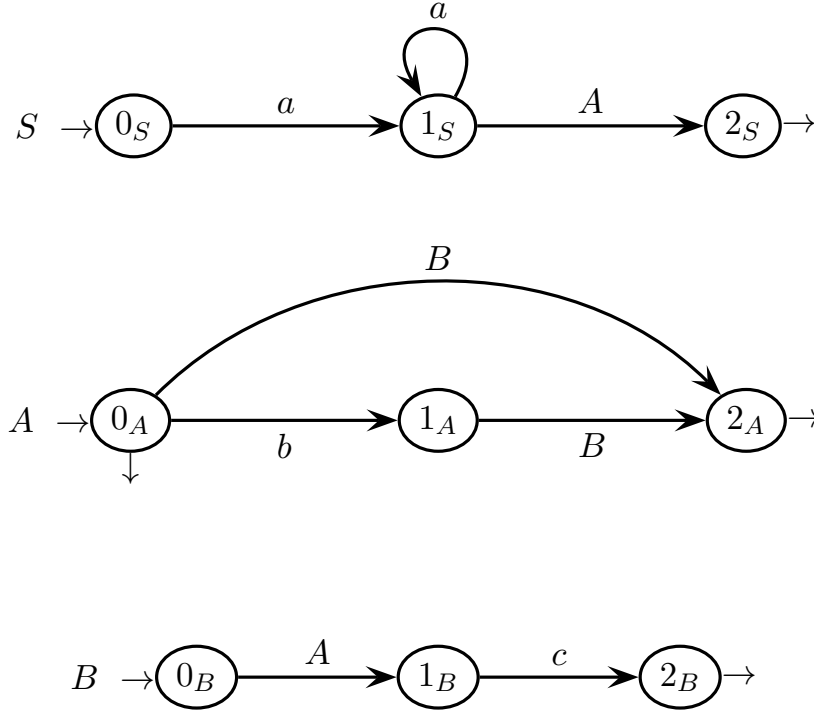
- (c) (optional) Extend the grammar to include the *object* constructed type, which is similar to a tuple, but allows each element to be named, and possibly optional, as in the following example:

```
type Student = { surname: string,
                  name: string,
                  person_code : number,
                  enrollment_code ? : number,
                  active : boolean
                }
```

where the terminal **?** denotes the optionality of the enrollment code field. Field names are modeled as identifiers $\langle \text{ID} \rangle$.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented by a machine net with the three-symbol nonterminal alphabet $V = \{S, A, B\}$ (axiom S) and the three-letter terminal alphabet $\Sigma = \{a, b, c\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

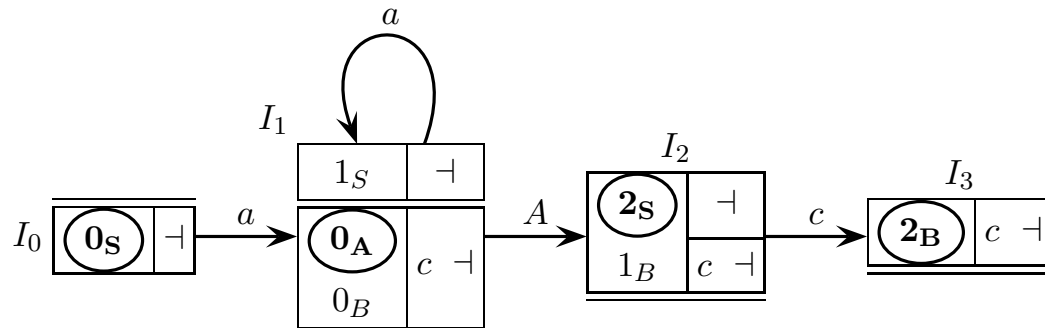
- (a) Write all the valid strings of grammar G of length ≤ 3 (less than or equal to three), and draw the syntax trees of the strings of length exactly three.
- (b) Draw the complete pilot of grammar G (part of it is already given) and say if grammar G is of type ELR. Explain your answer. Does the pilot have multiple transitions, and if so, which ones?
- (c) Find all the guide sets on the net of grammar G and prove that grammar G is **not** ELL(k), for any $k \geq 1$. Explain your answer.
- (d) (optional) Find a possible formulation of language $L(G)$ as a set of strings, like:

$$L(G) = \{ \text{string model(s)} \mid \text{characteristic predicate} \}$$

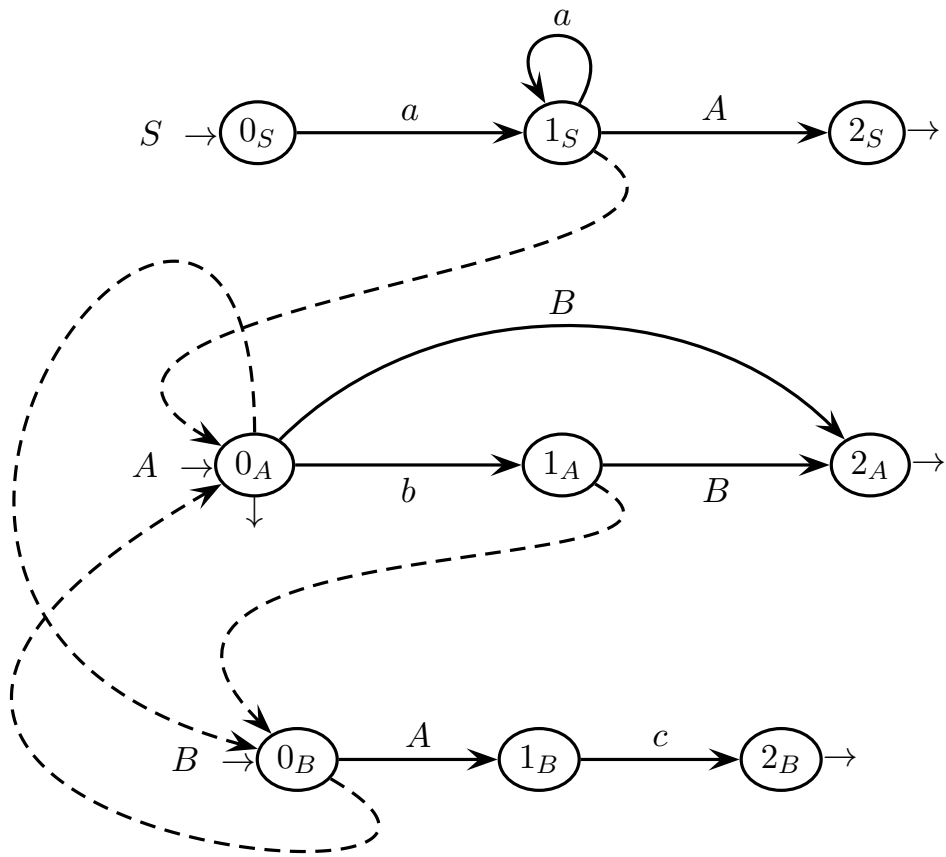
Say if language $L(G)$ is regular or not, and explain your answer.

here write the strings and draw the trees – question (a)

draw here the complete pilot and examine ELR – question (b)



write here the guide sets and examine ELL – question (c)

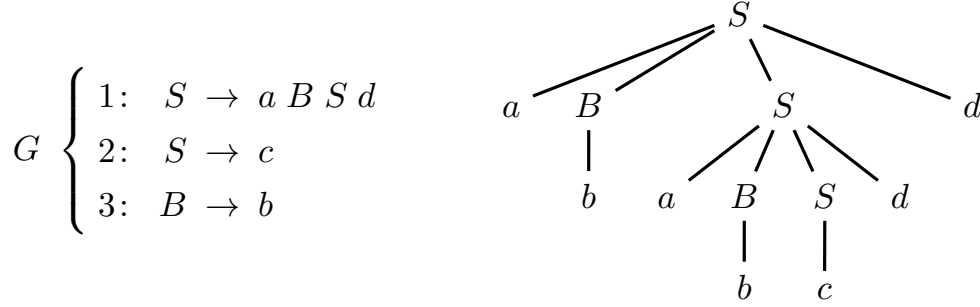


space for answering question (d)

free space for continuation or notes – any question

4 Language Translation and Semantic Analysis 20%

1. Consider the BNF grammar G below (left), with nonterminals S and B (axiom S), and terminals $\{a, b, c, d\}$, where the rules are numbered for reference:



The string $ababcbdd \in L(G)$ has the syntax tree shown above (right). It admits the leftmost derivation below (we subscript to the derivation symbol $\Rightarrow$ the number of the applied rule):

$$S \xRightarrow{1} a B S d \xRightarrow{3} a b S d \xRightarrow{1} a b a B S d d \xRightarrow{3} a b a b S d d \xRightarrow{2} a b a b c d d$$

and the rightmost derivation below:

$$S \xRightarrow{1} a B S d \xRightarrow{1} a B a B S d d \xRightarrow{2} a B a B c d d \xRightarrow{3} a B a b c d d \xRightarrow{3} a b a b c d d$$

Answer the following questions:

- (a) We intend to define a translation function $\tau_1: L(G) \rightarrow \{1, 2, 3\}^+$ that, for every string $x \in L(G)$, returns the *leftmost* derivation of x , encoded as the sequence of the rules applied to derive x . For instance:

$$\tau_1(ababcbdd) = 13132$$

Write a translation grammar or scheme G_1^T that computes translation τ_1 , with source grammar G , and draw the translation tree for the string $ababcbdd$.

- (b) We intend to define a translation function $\tau_2: L(G) \rightarrow \{1, 2, 3\}^+$ that, for every string $x \in L(G)$, returns the *rightmost derivation in reverse order* of x . For instance:

$$\tau_2(ababcbdd) = 33211$$

Write a translation grammar or scheme G_2^T that computes translation τ_2 , with source grammar G , and draw the translation tree for the string $ababcbdd$.

- (c) Is translation τ_1 invertible? Explain your answer.
- (d) Is translation τ_2 invertible? Explain your answer.
- (e) (optional) Is translation τ_1 deterministic? Explain your answer.
- (f) (optional) Is translation τ_2 deterministic? Explain your answer.

2. Consider the syntactic support G shown below, over the terminal alphabet $\Sigma = \{v, "\oplus", ":", "=", n\}$ and nonterminal alphabet $V = \{S, X, A, T\}$, defined by these rules (axiom S):

$$G \left\{ \begin{array}{ll} 1: & S \rightarrow X \\ 2: & X \rightarrow A ":" X \\ 3: & X \rightarrow \varepsilon \\ 4: & A \rightarrow v "=" T "\oplus" T \quad // \text{ assignment} \\ 5: & T \rightarrow v \\ 6: & T \rightarrow n \end{array} \right.$$

Grammar G represents a simple programming language, where a program is a sequence (X) of assignments (A). Variables are represented by terminal v . Two functions $f(v)$ and $l(t)$ are available to get the name of variable v and the line number corresponding to each terminal t , respectively. Numeric constants and arithmetic operators are represented by terminals n and $\oplus$, respectively.

We want to define an attribute grammar over the support G to enable the issuing of semantic warnings related to the use of variables that are not yet defined. To this end, we define the following attributes s , d and e :

- s is a right attribute inherited by the nonterminals X , A and T , which represents the set of variables defined as far as that point in the program; the value of s is a set of variable names
- d is a left attribute synthesized by nonterminal A , which represents the variable defined by the current assignment; the value of d is a set that contains a single variable name.
- e is a left attribute synthesized by all the nonterminals (S , X , A and T), which represents the detected errors, i.e., the lines containing utilizations of variables that are not previously defined; the value of e is a set of line numbers

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Write the semantic functions for the three attributes s , d and e above.
- (b) Write the computed values of the three attributes for the program below, which consists of three assignments:

```
a = 1 + 3;    // line 1
d = 2 + a;    // line 2  NB: variable a was previously defined
b = c + d;    // line 3  NB: d was previously defined, but not c
```

and decorate the parse tree of the program (see the next pages). The line numbers are not part of the program, but provide the values for function l .

- (c) Prove that the attribute grammar is $LL(k)$ (for some $k \geq 1$) and one-sweep.
- (d) Write the semantic procedure (in pseudocode) for analyzing nonterminal A .

table to be filled with the semantic functions – question (a)

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow X_1$	$s_1 :=$ $e_0 :=$
2:	$X_0 \rightarrow A_1 \text{ “;” } X_2$	$s_1 :=$ $s_2 :=$ $e_0 :=$
3:	$X_0 \rightarrow \varepsilon$	$e_0 :=$
// nonterminal A generates an assignment – it is associated to the attributes s , d and e // the attribute values of s , d and e for A must be shown in the program of question (b)		
4:	$A_0 \rightarrow v \text{ “=” } T_1 \text{ “} \oplus \text{” } T_2$	$s_1 :=$ $s_2 :=$ $d_0 :=$ $e_0 :=$
5:	$T_0 \rightarrow v$	$e_0 :=$
6:	$T_0 \rightarrow n$	$e_0 :=$

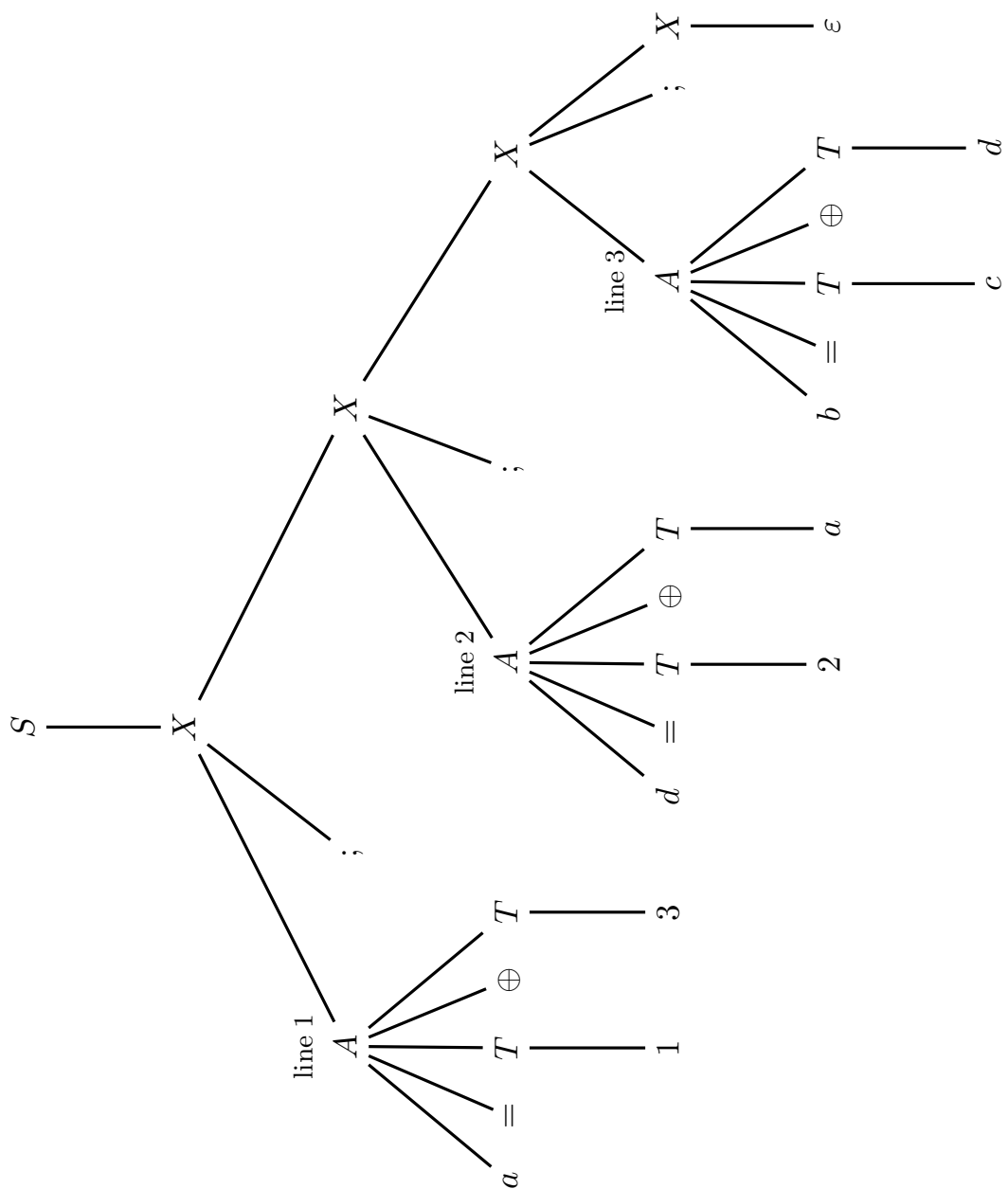
program and parse tree to be decorated with attribute values – question (b)

// write here the attribute values s0, d0, e0 for each assignment

a = 1 + 3; // line 1 s = d = e =

d = 2 + a; // line 2 s = d = e =

b = c + d; // line 3 s = d = e =



space for answering question (c)

space for answering question (d)

free space for continuation or notes – any question

free space for continuation or notes – any question